KARAKTER TABANLI "MARS KOLONİSİ YAŞAM DESTEK SİMÜLATÖRÜ" TEKNİK PROJE RAPORU

Hazırlayan: Muammer Eren Özünlü | **Öğrenci No:** 1260505013 | **Dil:** C Programlama Dili

1. Proje Özeti ve Durum Yönetimi Mimarisi

Bu projede, Mars'taki bir araştırma istasyonunda mahsur kalan bir mühendisin kurtarma ekibi gelene kadar **15 gün boyunca** hayatta kalmasını konu alan metin tabanlı bir simülasyon geliştirilmiştir. Tüm durum değişkenleri C dilinin temel veri tipleriyle tanımlanmış ve oyun boyunca aritmetik/mantıksal operatörlerle güncellenmiştir:

- **Yaşamsal ve Kaynak Değişkenleri:** `saglik` (0-100), `enerji` (0-100), `oksijen` (0-100), `su` (birim), `yedekParca` (adet).
- **İstasyon ve Zaman Değişkenleri:** `modulHasari` (0-100), `kilitMuhurlu` (0: Açık, 1: Mühürlü), `gun` (1-15) ve `tur` sayacı.
- **Rastgelelik Sistemi:** Programın başında `srand((unsigned int)time(NULL))` çağrılarak rastgele sayı üreteci başlatılmıştır. Olasılık içeren her komutta `zar = rand() % 100` ile 0-99 arası sayı üretilmekte ve karşılaştırıldığı eşik değer ekrana `Zar: 37 / Esik: 45 -> BASARILI` formatında yazdırılmaktadır.

2. Zorluk Seviyesi ve Komut Yönetimi (SWITCH-CASE & Fall-Through)

Simülasyon başladığında bir `do-while` döngüsü içinde ayrı bir `switch-case` yapısıyla zorluk seviyesi (**K: Kolay, N: Normal, Z: Zor**) seçilir. Geçersiz bir karakter girildiğinde `default` bloğu uyarı verir ve doğru seçim yapılana kadar döngü tekrarlanır. Zorluk seviyesi; başlangıç su/parça miktarlarını, keşif başarı eşiklerini ve kum fırtınasının tur sayısı ile hasar şiddetini doğrudan belirler.

Ana oyun döngüsü de `do-while` ile kurulmuş olup oyuncu komutları büyük/küçük harf duyarsız (*case-insensitive*) olacak şekilde **fall-through** tekniğiyle işlenmektedir. Tanımsız komutlarda `default` bloğu devreye girer ve tur sayacı artırılmaz (`turGecti = 0`):

```
switch (komut) {
    case 'g':
    case 'G':
        /* Kesif gorevi islemleri (kucuk veya buyuk harf ayni blogu calistirir) */
        break;
    default:
        printf("[UYARI] Tanimsiz bir komut girdiniz ('%c')! (Sure/tur harcamaz)\n", komut);
        turGecti = 0;
        break;
}
```

3. Aritmetik, Mantıksal İşlemler ve Karar Yapıları (IF - ELSE IF - ELSE)

3.1. Keşif Görevi (G) - Mantıksal Operatörler (&& ve ||)

Keşif komutu önce yeterli enerji kontrolü (`enerji < 15`) yapar; enerji yetersizse komutu iptal eder. Yeterliyse enerji 15, oksijen 10 azaltılır. Oyuncunun kondisyonu `(enerji > 40 && oksijen > 30)` mantıksal VE koşuluyla değerlendirilerek başarı eşiği belirlenir. Üretilen zara göre **su bulma**, **yedek parça bulma** veya **eli boş dönme** olmak üzere 3 farklı sonuç üretilir. Ayrıca `(zar < 10 || modulHasari > 70)` mantıksal VEYA koşulu sağlanırsa oyuncu kaza geçirerek 10 sağlık kaybeder.

3.2. Oksijen Üretimi (O) - Mod (%) Operatörü ve Aritmetik Dönüşüm

Her **2 birim su, 3 birim oksijene** dönüştürülür. Oyuncunun girdiği su miktarı mevcut sudan fazla olamaz. Girilen değerin çift veya tek olduğu **mod (%) operatörü** ile kontrol edilir; tek sayı girilirse 1 birimi depoda artık olarak bırakılır:

```
if (girilenSu % 2 == 0) {
    kullanilanSu = girilenSu;
    artikSu = 0;
} else {
    kullanilanSu = girilenSu - 1;
    artikSu = 1; /* 1 birim su depoda kalir */
}
uretilenOksijen = (kullanilanSu / 2) * 3;
su = su - kullanilanSu;
oksijen = oksijen + uretilenOksijen;
```

3.3. Tamir (T) - IF-ELSE IF-ELSE Kademeli Hasar Zinciri

Yedek parça yoksa (`yedekParca <= 0`) tamir iptal edilir. Modül hasarı `if - else if - else` zinciri ile 3 kademeye ayrılmıştır: **Hafif Hasar (1-30)** kademesinde 1 parça harcanıp 15 hasar onarılır; **Orta Hasar (31-70)** kademesinde 2 parça varsa 30, 1 parça varsa 20 hasar onarılır; **Ağır Hasar (71-100)** kademesinde ise 3 parçaya kadar kapsamlı onarım yapılarak 45 birime kadar hasar giderilir.

3.4. Dinlenme (U) - İç İçe IF (Nested IF) Yapısı

Dinlenme komutunda enerji (+25/+30) artırılırken oksijen solunumu (-5) devam eder. Sağlık artışı ise **iç içe `if`** yapısıyla oksijen seviyesine bağlanmıştır: Oksijen 20'nin altındaysa hipoksi nedeniyle sağlık artmaz, aksine 10 birim azalır:

```
if (enerji < 100 || saglik < 100) {
    enerji = enerji + enerjiArtisi;
    if (oksijen < 20) {
        saglik = saglik - 10; /* Ic IF: Oksijen < 20 ise saglik azalir */
    } else {
        saglik = saglik + saglikArtisi;
    }
}
```

3.5. Durum Raporu (D) - Ayrı Uyarılar ve Tek Koşullu Normal Sistem Mesajı

Durum raporu tur harcamaz. Kritik eşiklerin altındaki her değer için ayrı `if` bloklarıyla uyarı verilir (`oksijen < 25`, `saglik < 30`, `modulHasari > 70`, `enerji < 20`, `su < 2`). Tüm değerler güvenli aralıktaysa `&&` ile birleştirilmiş **tek bir koşulla** onay mesajı yazdırılır:

```
if (oksijen >= 25 && saglik >= 30 && modulHasari <= 70 && enerji >= 20 && su >= 2) {
    printf("  [SISTEM DURUMU] TUM SISTEMLER NORMAL\n");
}
```

4. Döngü Mekanizmaları (FOR ve DO-WHILE Zorunlulukları)

4.1. FOR Döngüsü ile Kum Fırtınası Simülasyonu (K) - Mod, Continue ve Break

Kum fırtınası zorluk seviyesine göre (Kolay: 3, Normal: 5, Zor: 7 tur) bir `for` döngüsüyle yürütülür:

- **Çift Tur Kontrolü:** `if (i % 2 == 0)` koşuluyla çift numaralı turlarda rüzgar şiddetlenir ve hasar çarpanı 2 katına (`carpan = 2`) çıkar.
- **`continue` Kullanımı:** Oksijen sızıntısı olayında hava kilidi önceden mühürlenmişse (`kilitMuhurlu == 1`), sızıntı hasarı `continue` ile o tur için atlanır.
- **`break` Kullanımı:** Fırtına sırasında sağlık veya oksijen 0'a düşerse döngü `break` komutuyla derhal sonlandırılır ve fırtına özeti ekrana basılır.

4.2. DO-WHILE Döngüsü ile Hava Kilidi Şifresi (H)

Hava kilidi şifresi sabit değildir; o anki gün sayısından `dogruSifre = 'A' + (gun % 26)` aritmetik formülüyle türetilir. Küçük harf girişleri aritmetik olarak büyük harfe çevrilir. Oyuncunun 3 deneme hakkı vardır; her yanlış tahminde enerji 5 azalır ve alfabetik sıralamaya göre **"İLERİ GİT"** veya **"GERİ GİT"** ipucu verilir. Döngü koşulu, hem şifrenin yanlış olmasını hem de hakkın bitmemiş olmasını **tek bir mantıksal ifadede** kontrol eder:

```
do {
    /* Harf okuma, buyuk harfe cevirme ve dogrulama islemleri */
} while (buyukGirilen != dogruSifre && kalanHak > 0);
```

5. Sınırlandırma, Zaman Akışı ve Oyun Sonu Puan Hesabı

- **Sınırlandırma (0-100):** Her komuttan sonra `saglik`, `enerji`, `oksijen` ve `modulHasari` değişkenleri `if` yapılarıyla kontrol edilerek 0'ın altına düşmesi ve 100'ü aşması engellenir.
- **Zaman Akışı:** Geçerli her komut `tur` sayacını 1 artırır. `if (tur % 3 == 0)` koşulu sağlandığında yeni bir gün başlar; gün başında oksijen 8 azalır ve `modulHasari > 70` ise ek olarak 5 sağlık kaybedilir.
- **Oyun Sonu ve Puan Formülü:** Sağlık veya oksijen 0'a düşerse oyun kaybedilir; 15. gün tamamlanırsa kurtarma ekibi gelir ve oyun kazanılır. Oyun hangi şekilde biterse bitsin (Kazanma, Kaybetme veya Q ile çıkış) sonuç raporu ve aşağıdaki aritmetik formülle hesaplanan puan yazdırılır:
  `puan = (gun * 100) + saglik + oksijen - modulHasari;`
  Katsayı Gerekçelendirmesi: Hayatta kalınan her gün en kritik başarı ölçütü olduğu için `100` katsayısı ile çarpılmış, kalan sağlık ve oksijen pozitif katkı olarak eklenmiş, istasyonda bırakılan modül hasarı ise ceza puanı olarak düşülmüştür.

Proje GitHub Kaynak Kodu Bağlantısı (Public Repository):
https://github.com/merenozunlu/mars-yasam-destek-simulasyonu